

Reusable UI patterns

Module 3 - Lesson 3

Overview

Great React code is composed of small, focused components that can be reused everywhere. Consistency, clear props, and composition over inheritance keep the codebase maintainable.

Key Concepts

- Compose primitives into larger components instead of copying and pasting markup.
- Design component APIs around clear props and sensible defaults.
- Use children prop and slots to let parents inject content.
- Keep components presentational when possible; lift logic into hooks or containers.
- Name components for what they render and keep files small.
- Extract repeated logic into custom hooks (useLocalStorage, useDebounce, useFetch).

Example

```
function Card({ title, children, actions }) {  
  return (  
    <div className="card">  
      <h3>{title}</h3>  
      <div>{children}</div>  
      {actions}  
    </div>  
  );  
}
```

Summary

Reusable UI patterns come from composition, clear props, and extracted hooks. Small focused components with flexible children render consistently and are easy to maintain.

Practice Checklist

- Refactor three similar blocks in your app into one reusable component.
- Extract a useDebounce hook and use it in a search input.
- Review a component's props and give them sensible defaults.